



GO-LIVE GUIDE · VERSION 1.0 · 6 OCTOBER 2026

Veejay Foods

From today's working build to a live, public app: every step, the accounts and tools you need, who does what, and what can and can't be done for you.

Prepared for: Veejay Foods — launch area Abuja Municipal (AMAC), pickup only
Covers: customer app (iOS & Android) · API & database · staff / admin dashboard
Companion files: Design & delivery plan · Architecture notes · repository README

1 • Where we are today

The app is a **working, tested build with sample data**. It is not yet running on the internet, and it has never handled a real payment. The distance to launch is mostly accounts, approvals, real content and real-device testing — not new features.

Done and verified

- Customer app, API and staff/admin dashboard are built (all nine planned capabilities).
- 67 automated backend tests pass on a real PostgreSQL database — payments webhook, order states, QR pickup, sessions, refunds, slots.
- App type-checks and bundles for Android; dashboard builds; full customer and staff flows were driven in a browser against the live API.
- Production start-up verified: `npm run build` then `npm start` serves the API; `.env` files load; an initial database migration applies cleanly to an empty database with zero drift.
- A production-safe command to create the first admin (`admin:create`) — separate from the sample-data seed.
- Brand applied (name, logo, colours); launch area set to Abuja.

Not done yet

- No hosting, domain or live database.
- No Paystack live account; real payments, refunds, SMS and push notifications have **never been exercised** (tests use stand-ins).
- Never run on a physical iPhone or Android phone.
- Outlets, menu, prices and photos are **samples** (Wuse II, Garki, Jabi).
- Photos are stored on the server's disk (lost on redeploy).
- No in-app account deletion, privacy policy or terms (both app stores require them).
- No store accounts, store listings, or mobile build configuration.
- No monitoring, alerting or restore-tested backups.

Bottom line

Yes, the app can be deployed, launched, handed over and opened to the public. Expect roughly **4-8 weeks** from the day accounts are opened. The hands-on work is about 2-3 weeks; the rest is waiting for Paystack, SMS sender-ID and app-store approvals, which you do not control. (These are estimates, not guarantees.)

How to read this guide

Every step carries an owner tag:

YOU the developer/agency delivering the project **CLIENT** Veejay Foods (owner/admin; legal and payment identity)
CLAUDE work I can do in this repository **TOGETHER** needs both

Each phase ends with a **“Done when”** test. Don't start the next phase until it passes — most launch problems come from skipping one.

2 · The route at a glance

#	PHASE	GOAL	MAINLY	TYPICAL TIME
0	Decisions & content	Real outlets, menu, prices, refund rules, staff list, launch date	CLIENT	1 wk (parallel)
1	Accounts & access	Every account exists under the client's name; long-lead approvals started on day 1	TOGETHER	1-3 wks (waiting)
2	Production hardening	Close the code gaps that would block deployment or store approval	CLAUDE YOU	1-2 wks
3	Deploy	Staging and production: API, database, dashboard, domain, HTTPS	YOU	3-5 days
4	Payments, SMS, push	Test-mode drills, then one real payment and refund	TOGETHER	~1 wk
5	Mobile builds & device testing	Installable builds tested on real phones	YOU CLIENT	1-2 wks
6	Soft launch	Invite-only at one or two outlets; staff trained	CLIENT YOU	2-3 wks
7	Store submission & public launch	Approved on App Store and Google Play; opened to the public	TOGETHER	1-2 wks (review)
8	Handover & hypercare	Client owns and can run everything; support period	TOGETHER	2 wks

Indicative timeline (weeks from the day accounts are opened)

	W1	W2	W3	W4	W5	W6	W7	W8
0 Decisions & content								
1 Accounts & approvals (waiting)								
2 Production hardening								
3 Deploy (staging → prod)								
4 Payments / SMS / push drills								
5 Builds & device testing								
6 Soft launch								
7 Store review & public launch								
8 Handover & hypercare								

The critical path is approvals, not code

Start Phase 1 on **day one** and in this order: **(1)** Paystack business verification, **(2)** SMS sender-ID registration, **(3)** Apple and Google developer accounts. Each can take days to weeks and none can be rushed from our side. Everything else can run while you wait. If any of these stalls, the launch date moves — nothing else will.

3 · What I can do, what I can't, and what only you or the client can do

I'm Claude, working inside this code repository. This is the honest boundary, so nobody plans around something that can't happen.

✓ I can do

- Write and change code in the app, API and dashboard; add tests; run builds and the test suite here.
- Create deployment files: Docker/host configuration, `eas.json`, CI workflows, environment templates.
- Add the missing production features: account deletion, cloud image storage, restricted CORS, reviewer test login, error monitoring.
- Write runbooks, checklists, staff quick-cards, store-listing text, and a *draft* privacy policy and terms.
- Prepare menu/outlet import from a spreadsheet you supply.
- Generate screenshots for store listings from the web build.
- Walk you through each console click-by-click and read logs or error messages you paste.
- Re-run the whole test suite and re-generate this document as things change.

✗ I can't do

- Create or verify accounts (Paystack, Apple, Google, SMS, hosting). They need identity, bank details, documents and payment cards.
- Log in to the client's accounts, or hold their secrets. (Don't paste keys into chat.)
- Deploy to a host from this sandbox — I have no credentials and limited outbound access. I can prepare everything; a person runs the deploy.
- Run the app on a physical iPhone/Android, test real SMS delivery, real push, or a real card or bank transfer.
- Submit to the App Store / Play Store or talk to reviewers; I can write the answers.
- Speed up or guarantee any third-party approval.
- Give legal advice (privacy law, tax, contracts) — drafts need a lawyer's review.
- Supply the real menu, prices, photos or addresses.
- Monitor the live system after this conversation ends, or be on call.
- Make business decisions (refund policy, opening hours, pricing).

★ Only you / the client can

- **CLIENT** Legal identity: business registration documents, signing agreements, bank settlement account.
- **CLIENT** Real content: outlets, menu, prices, photos, support contact.
- **CLIENT** Business rules: refunds, order cut-offs, who is staff/admin.
- **CLIENT** Paying for accounts, SMS credit and hosting.
- **YOU** Run deployments, builds and submissions with the client's credentials.
- **YOU** Hold the phones for device testing (one Android, one iPhone minimum).
- **TOGETHER** Train staff; run the soft launch; sign off acceptance.

Practical working rule

Keep all secrets in a **password manager** owned by the client and typed straight into the host's settings page. When I need to know whether something is set up correctly, share the *names* of the settings and the *error text* — never the values.

4 · Resources and accounts needed

Open these under the **client's** ownership (company email, company payment card). Add the developer as a team member — never the other way round — so handover is simply removing access.

RESOURCE	WHY IT'S NEEDED	OWNER	COST TYPE (confirm current prices)	LEAD TIME
Business documents	Needed by Paystack (and Apple for an organisation account). Exact documents depend on how the business is registered.	Client	—	Gather in week 1
Paystack business account	Takes card, bank-transfer and USSD payments; sends webhooks; processes refunds. Provides test keys immediately and live keys after approval.	Client	Per-transaction fees	Days to weeks
Settlement bank account	Where Paystack pays out the money.	Client	—	Same as above
SMS provider (code supports Termii)	Sends login codes. Sender-ID registration and OTP routing to DND-registered numbers must be confirmed with the provider.	Client	Prepaid per SMS	Days to weeks
Apple Developer Program	Publish on the App Store; push-notification keys. Organisation enrolment typically needs a D-U-N-S number.	Client	Yearly fee	Days to 2+ weeks
Google Play Console	Publish on Google Play. Identity verification applies; new accounts may have to run a closed test with a minimum number of testers first — check current Play rules.	Client	One-time fee	Days; test period may add ~2 wks
Expo (EAS) account	Cloud builds for iOS/Android (no Mac needed), push credentials, store submission.	Client	Free tier; paid for priority	Minutes
Hosting + managed PostgreSQL	Runs the API and the database with automatic backups.	Client	Monthly	Minutes
Domain name + DNS access	e.g. <code>api.</code> for the API and <code>staff.</code> for the dashboard; HTTPS certificates.	Client	Yearly	Hours
Image storage (S3-compatible)	Menu images and complaint photos that survive redeloys.	Client	Usage-based	Minutes
GitHub organisation	Owns the code; the developer is added as a member.	Client	Free / paid	Minutes
Error monitoring (e.g. Sentry) + uptime check	Know about crashes and outages before customers do.	Client	Free tier available	Minutes
Support email / WhatsApp	Customer-facing contact in the app and store listings; business email for Paystack receipts.	Client	—	Hours
Legal review	Privacy policy, terms of use, Nigerian data-protection (NDPA) obligations.	Client	Professional fee	1–2 wks
Real menu, photos, outlet details, logo files	Replace all sample data (photos: square, at least 1000 px).	Client	—	Week 1–2
Counter devices	One Android tablet/phone per outlet (Chrome) for the staff board and QR scanning; stable internet.	Client	One-time	Days
Test phones & money	≥1 iPhone, ≥2 Android (one low-end) and small real payments for the final drill.	You	—	Before Phase 5

5 · Tools and software needed

TOOL	USED FOR	WHO INSTALLS
Node.js 22 + npm	Build and run the API, dashboard and app tooling.	You (and the host)
Git + GitHub	Code history, review, CI (tests run automatically on every push).	You
EAS CLI — <code>npm i -g eas-cli</code>	Cloud builds, credentials and store submission for the mobile app. <i>No Mac required.</i>	You
Prisma CLI (via <code>npx</code>)	Applies database migrations in each environment.	You (host build step)
A database client (e.g. <code>psql</code> , TablePlus)	Inspect data and verify backups (read-only habits in production).	You
Chrome on staff devices	The dashboard and camera QR scanner (camera needs HTTPS).	Client
TestFlight (iOS) & Play internal testing (Android)	Distribute test builds to real phones before public release.	You / testers
Password manager	Store every secret and account login; client-owned shared vault.	Client
curl / Postman	Check <code>/health</code> and API behaviour in each environment.	You
Sentry + an uptime monitor	Crash reports and “is it up?” alerts.	You / Client
Spreadsheet (Excel / Sheets)	Client fills the menu and outlet template that I can turn into an import.	Client

Hosting choice (decision needed in Phase 0)

OPTION	GOOD FOR	WATCH OUT FOR
A. Managed app platform + managed Postgres (e.g. Render, Railway, Fly.io)	Fastest, least upkeep, automatic HTTPS and deploys from GitHub. Recommended to start.	Data centre is usually in Europe/US — latency is fine for this app, but ask the lawyer about cross-border personal-data transfer.
B. Cloud virtual machine + managed database (e.g. AWS, DigitalOcean, Azure)	More control; some providers have an Africa region.	You run and patch the server; more to set up and monitor.
C. Nigerian hosting provider	Local data residency.	Maturity and managed-database features vary; check backups, TLS and support carefully.

Whichever is chosen: the client’s account, daily backups with a tested restore, automatic HTTPS, and a region decision made together with the privacy review.

PHASE 0 Decisions & content

Owner: **CLIENT** · about 1 week, runs in parallel with Phase 1 · I provide the spreadsheet templates.

- 1 Confirm the brand assets** **CLIENT**
Legal business name, logo in vector form (SVG/AI/PDF) plus a high-resolution PNG, brand colours. (The app currently uses a cropped logo image and colours sampled from it.)
- 2 List the real Abuja pickup points** **CLIENT**
For each: display name, street address, opening hours per day, preparation time, how many orders the kitchen can handle per 30-minute slot. Start with one or two outlets for the soft launch.
- 3 Prepare the menu** **CLIENT**
Categories, item names, descriptions, prices in ₦, combos/meal plans with what's included, and one square photo per item. Items that must not be orderable at a given outlet.
- 4 Decide the refund and complaint policy** **CLIENT**
Who may approve refunds, whether partial refunds are allowed, how long customers have to report a problem, and what happens to no-shows (paid orders never collected).
- 5 Decide ordering rules** **CLIENT**
Online ordering hours, how far ahead customers can book (the app currently offers today and the next 3 days), minimum prep time, cut-off before closing.
- 6 Name the people** **CLIENT**
Admins and staff: full name, phone number, which outlet. They sign in with an SMS code, so the number must be theirs.
- 7 Choose a support contact and a launch date** **CLIENT**
A phone/WhatsApp/email shown in the app and store listings; a target date and the first 20–50 invitees for the soft launch.
- 8 Pick the hosting option** **TOGETHER**
See section 5. A one-line written decision is enough.

Done when: the client has signed off the outlet list, menu, policies and staff list in writing (email is fine), and the hosting option is chosen.

PHASE 1 Accounts & access

Owner: **TOGETHER** · 1–3 weeks of mostly waiting · **start the first three steps on day one**

- 1 Paystack: start business verification** **CLIENT**
Register at paystack.com, complete the business profile, upload the requested documents, add the settlement bank account. In parallel, copy the **test** keys (Settings → API Keys & Webhooks) so development isn't blocked. Live keys appear only after approval.
- 2 SMS provider: open the account and register a sender ID** **CLIENT**
Create the account (the code supports Termii), fund the wallet, submit the sender ID (e.g. "Veejay"). **Ask them in writing** which route delivers OTPs to numbers on the Do-Not-Disturb list — the current code uses the provider's generic channel, and that may need changing.
- 3 Developer accounts: Apple and Google** **CLIENT**
Enrol the business in the Apple Developer Program (organisation accounts usually require a D-U-N-S number — request it early) and create the Google Play Console account with identity verification. Check Google's current requirement for a closed-testing period on new accounts.

4 GitHub: put the code under the client **TOGETHER**

Client creates a GitHub organisation; transfer or fork the repository into it; add the developer as a member. Merge the working branch (`ccr-162c3e6b-4vdvog`) into the main branch via a reviewed pull request.

5 Expo (EAS) account **CLIENT**

Create an Expo organisation account, add the developer. Later, `eas init` creates the project ID that push notifications need.

6 Hosting, database, domain, storage **CLIENT**

Create the host account and a managed PostgreSQL instance with daily backups; buy or point the domain; create an S3-compatible bucket (private).

7 Create the secrets vault **CLIENT**

A shared password-manager vault. Rule: secrets live **only** there and in the host's settings — never in chat, email, screenshots or Git.

8 Monitoring **YOU**

Create an error-monitoring project and an uptime check (it will watch `/health`) under the client's email.

Done when: every account in section 4 exists under the client's ownership; the developer has team-member access; Paystack, SMS sender-ID and the developer-account applications are submitted and you know their expected dates.

If an approval stalls

Keep building and testing on test keys and the staging environment. Don't go live on a workaround (e.g. taking payments into a personal account) — it breaks refunds, accounting and the store's payment rules.

PHASE 2 Production hardening

Owner: **CLAUDE** writes it · **YOU** review and merge · 1-2 weeks · can start now

These close the gaps between “works in testing” and “safe to put real customers on”. Effort is an AI-assisted estimate in focused working days.

#	WORK ITEM	WHAT IT DOES / WHY IT MATTERS	STATUS	EFFORT
1	Fix <code>npm start</code> / build	The production start command previously crashed; now builds to <code>dist/</code> and runs.	Done	—
2	Load <code>.env</code> in dev and seed	Settings files were silently ignored.	Done	—
3	Database migrations	Initial migration (16 tables) applied to an empty DB with zero drift; CI now runs it every push. Future schema changes ship as new migrations.	Done	—
4	<code>admin:create</code> command	Creates the first admin only — the sample-data <code>seed</code> must never be run in production.	Done	—
4b	Strict production guard	Start-up refuses placeholder/short secrets and console SMS; <code>prisma</code> moved to main dependencies so <code>npm run migrate</code> works on hosts that install production packages only. Proven end-to-end on a clean install.	Done	—
5	Deploy configuration	Dockerfile / host build & start commands, health check, graceful shutdown, <code>.env</code> checklist.	To do	0.5 d
6	Restrict CORS	Today any website can call the API from a browser. Limit to the dashboard’s address.	To do	0.25 d
7	Cloud image storage	Menu images and complaint photos go to a private bucket (served via short-lived links) instead of server disk.	To do	1 d
8	Account deletion	In-app “Delete my account” + server endpoint (anonymise personal data, keep order/payment records as required). Both app stores require it.	To do	1 d
9	Privacy policy & terms pages	Hosted pages linked from the app and store listings. I draft; a lawyer reviews.	To do	0.5 d + legal
10	Store-reviewer login	Reviewers can’t receive a real SMS. Add a designated test phone with a fixed code, enabled only by a setting, and removed after approval.	To do	0.25 d
11	Mobile build configuration	<code>eas.json</code> profiles (preview / production), bundle IDs, version numbers, deep-link scheme check, push-credential setup notes.	To do	0.5 d
12	Error monitoring & logging	Crash reports from app and API; structured logs; alert on webhook failures.	To do	0.5 d
13	SMS route for DND numbers	Switch the OTP channel once the provider confirms the correct route.	To do	0.25 d
14	Menu/outlet import	Load the client’s spreadsheet instead of typing items one by one.	Optional	0.5 d
15	Runbook & staff quick-card	“If X happens, do Y” for the owner; a one-page counter guide for staff.	To do	1 d
16	Dependency & security pass	Update packages, run an audit, review security headers and rate limits.	To do	0.5 d

Known limits to keep in mind

- **Single API instance.** The 60-second order-expiry job and the login rate limiter live in the server’s memory. That is fine for one instance; before running several, the rate limiter needs a shared store.
- **Pickup only.** No delivery, promo codes or loyalty at launch.

Done when: items 5-13 and 15-16 are merged, CI is green, and the test suite passes after the changes.

PHASE 3 Deploy: staging, then production

Owner: **YOU** (with the client's accounts) · 3–5 days · I prepare the files and can debug from error text

Build **staging first** — an identical copy with its own database and Paystack *test* keys — then repeat for production. Never test on production data.

1 Create the database **YOU**

Managed PostgreSQL, daily backups on, access limited to the API. Copy the connection string into the password manager.

2 Generate three different secrets **YOU**

One each for sessions, login codes and QR codes:

```
openssl rand -base64 48 # run three times; store each in the vault
```

3 Create the API service from the repository **YOU**

Point the host at `veejay-foods/backend`. Set the settings listed in Appendix A, then:

```
Build command : npm ci && npm run build
Start command  : npm run migrate && npm start
Health check   : GET /health (returns {"ok":true})
```

In production (`NODE_ENV=production`) the server refuses to start with placeholder or short secrets, the test Paystack placeholder, or console SMS — that's intentional. For the staging copy use `NODE_ENV=staging` with its own random secrets.

4 Attach the domain **TOGETHER**

Point `api.<domain>` at the service and enable automatic HTTPS. Set `PUBLIC_API_URL` to that address.

5 Create the first admin — not the seed **YOU**

```
node dist/create-admin.js 0803XXXXXXX "Owner Name"
```

Never run `npm run seed` in production: it adds sample outlets and menu items.

6 Publish the staff dashboard **YOU**

```
cd veejay-foods/admin
VITE_API_URL=https://api.<domain> npm run build
# upload the dist/ folder to any static host at staff.<domain>
```

It must be on **HTTPS**, or browsers will block the camera used for QR scanning.

7 Enter the real outlets and menu **TOGETHER**

Sign in to the dashboard as the admin, then fill Outlets (hours, prep time, capacity), Menu (categories, items, combos, photos) and Staff (people from Phase 0).

8 Smoke test **YOU**

```
curl https://api.<domain>/health → {"ok":true}; sign in on the dashboard; confirm the outlets list loads from /outlets; confirm the uptime monitor is green.
```

Done when: staging and production both answer on HTTPS, the owner can sign in to the dashboard on each, and backups are confirmed on.

Back up before you trust the backups

Restore one backup into a scratch database and open it. A backup you have never restored is a hope, not a backup. Do this once before launch and again after handover.

PHASE 4 Payments, SMS and push notifications

Owner: **TOGETHER** · about 1 week · this is the first time real third-party services touch the app

1 Wire up Paystack in **test** mode (staging) **YOU**

Put the test secret key in `PAYSTACK_SECRET_KEY`. In the Paystack dashboard, set the webhook URL to `https://api.<staging-domain>/webhooks/paystack` (test and live modes keep separate settings — confirm in the dashboard). Set `PAYSTACK_EMAIL_DOMAIN` to a domain the client owns.

2 Run the payment drills **YOU**

Use the test cards/accounts listed in Paystack's own documentation. Every row must pass:

DRILL	EXPECTED RESULT
Successful card payment	Order becomes Placed ; push received; QR appears
Declined / failed payment	Order stays unpaid; can retry; slot released after 15 min
Close the browser mid-payment	Order screen reconciles itself and shows the right state
Re-send the same webhook from Paystack's dashboard	No duplicate order event, no second notification
Bank transfer and USSD (where test mode supports them)	Order placed once the payment is confirmed
Admin approves a full refund, then a partial refund	Refund created; status moves to Success when Paystack confirms
Paystack refund refused (e.g. already refunded)	Complaint reopens; admin sees a clear error

3 Switch SMS on **YOU**

Set `SMS_PROVIDER=termii`, `TERMII_API_KEY`, `TERMII_SENDER_ID`. Request login codes to numbers on **MTN, Airtel, Glo and 9mobile** and to a number on the DND list. Note delivery time. If any network fails, stop and fix with the provider before continuing.

4 Push notifications **YOU**

Run `eas init` and put the project ID in the app config. Upload the Android (FCM) credentials and let EAS create the Apple push key, following Expo's current instructions. Remote push does **not** work in Expo Go on Android — test on an installed build (Phase 5).

5 Go live with payments — only after Paystack approves **TOGETHER**

Put the **live** key in the **production** settings only; set the live webhook URL; then make one real small purchase from a staff phone and refund it. Check the money actually arrives, leaves, and appears in the Paystack dashboard and bank statement.

Done when: every drill row passes on staging; OTPs arrive on all four networks; one real payment and one real refund succeeded on production.

PHASE 5 Mobile builds & real-device testing

Owner: **YOU** + **CLIENT** testers · 1-2 weeks · I've never run the app on a phone — this phase finds what that misses

1 Create test builds **YOU**

```
cd veejay-foods/app
eas login && eas init
EXPO_PUBLIC_API_URL=https://api.<staging-domain> # set in the EAS build profile
eas build --platform android --profile preview # installable APK link
eas build --platform ios --profile preview # needs the Apple account
```

(The `eas.json` profiles are Phase 2, item 11.)

2 Install on real phones **TOGETHER**

At least **one iPhone, two Android phones (one low-end or older)**. Distribute with TestFlight and Play internal testing.

3 Work through the device checklist **TOGETHER**

- ☐ First launch → enter phone → SMS code arrives → signed in
- ☐ Menu loads with images; category chips scroll and jump; combo shows what's included
- ☐ Cart survives closing and reopening the app
- ☐ Checkout: day and time slots correct for Abuja time; full/past slots hidden
- ☐ Paystack page opens, payment completes, **app returns automatically**
- ☐ Switch to airplane mode mid-order, then back: no sign-out, no duplicate order
- ☐ Force-quit during payment: reopening shows the correct order state
- ☐ Push arrives for Placed / Preparing / Ready / Collected, app open and app closed; tapping opens that order
- ☐ QR is readable by the counter scanner at low and high screen brightness
- ☐ Report a problem: camera and gallery photo both upload
- ☐ Large-text and dark-mode phones are still readable; small screens don't cut off buttons
- ☐ Poor network (throttled): spinners and errors are understandable; nothing double-submits

4 Counter hardware test **CLIENT**

On the real counter device, open the dashboard, scan a QR from a customer phone in real lighting, and use the order-code fallback.

5 Fix, rebuild, repeat **CLAUDE** **YOU**

Send me each bug with the phone model, steps and a screenshot; I fix and add a test where it makes sense.

Done when: the checklist passes on every test phone and the counter device, with no open “must-fix” issues.

PHASE 6 Soft launch (invite-only)

Owner: **CLIENT** + **YOU** · 2–3 weeks · one or two outlets, 20–50 invited customers

1 Cut over to real data **TOGETHER**

Production has only real outlets, menu and staff (no sample items). Prices, photos and hours double-checked by the client.

2 Train the staff (about 1 hour per outlet) **TOGETHER**

Sign in; read the board; New → Preparing → Ready; scan and hand over; use the order-code fallback; what to say to a customer with a problem; how to mark an item sold out. Leave the one-page quick-card at the counter.

3 Release the builds to invitees **YOU**

TestFlight / Play internal or closed testing, with a short “how to order” message.

4 Review daily **TOGETHER**

Every morning for the first two weeks: orders vs. payments in Paystack (must match), refunds, complaints, login-code failures, app crashes, slot fullness, counter wait times.

5 Tune the numbers **CLIENT**

Adjust preparation time, slot length and orders-per-slot from the Outlets tab until the kitchen is comfortable.

Proposed go / no-go checks before the public launch

Targets to agree with the client — these are suggestions, not measured facts.

- ☐ **Zero** lost payments: every successful Paystack charge has a matching order or refund
- ☐ Login codes arrive within an agreed time on all four networks
- ☐ No unresolved crash reports; no must-fix bugs
- ☐ Refunds work end to end, and staff are confident handing over by QR
- ☐ Feedback from invitees reviewed; top issues fixed

Done when: the client signs a written “go” after the checks above.

PHASE 7

Store submission & public launch

Owner: **TOGETHER** · 1-2 weeks, mostly review time · store rules change — check each store's current requirements before submitting

1 Prepare the store listing **CLAUDE** **CLIENT**

App name, short and long descriptions, category (Food & Drink), keywords, icon, phone screenshots (I generate them from the app), support email/URL, privacy-policy URL, age rating, and each store's privacy questionnaire (Apple "App Privacy" labels, Google "Data safety" form). I draft the text; the client approves it.

2 Give the reviewers a way in **YOU**

Enable the reviewer test login (Phase 2, item 10) and put the phone number and code in the store's "review notes", with a line explaining that payment is by Paystack for physical food collected at an outlet. **Disable it after approval.**

3 Submit to Apple **YOU**

Production build → App Store Connect (**easy submit** or upload) → add to the version → submit for review. Review usually takes a few days but can be longer; answer reviewer questions the same day. Typical rejection reasons to pre-empt: missing in-app account deletion, broken reviewer login, vague permission descriptions.

4 Submit to Google Play **YOU**

Production build → Play Console → complete the Data safety form, content rating and (if required for a new account) the closed-testing period → release to production. Use a **staged rollout** (e.g. 10% → 50% → 100%) if offered.

5 Public launch day **TOGETHER**

Switch the listings live; confirm the production API is healthy; print the in-store QR poster pointing to the store link; announce on the brand's social media; keep the owner and developer reachable all day.

6 Watch the first 100 orders **TOGETHER**

Check every one against Paystack and the kitchen's experience; fix anything odd within a day.

Done when: the app is live and downloadable on both stores, and the first 100 orders reconcile cleanly.

Plan for a rejection

A first-time rejection is common and usually fixable in a day or two. Leave a week of buffer between "submitted" and any hard public date (a promotion, a poster print run).

PHASE 8

Handover & hypercare

Owner: **TOGETHER** · 2 weeks of close support after launch, then a maintenance arrangement

Ownership transfer checklist

- ☐ Client is the **Owner / Account Holder** on: GitHub, Paystack, SMS provider, Apple, Google Play, Expo, hosting, database, domain/DNS, storage, monitoring
- ☐ Developer access reduced to “team member” (or removed) on each — and **production secrets rotated** so the developer no longer holds working copies
- ☐ Password-manager vault contents verified by the client (every login and key present and working)
- ☐ Store accounts: client’s email for notices; two-factor on for every account; a second admin added in case the owner is unavailable
- ☐ Renewal calendar created: Apple yearly fee, domain, certificates, SMS credit top-ups, hosting plan
- ☐ Final backup **restore test** performed with the client watching

What the client receives

- **Runbook** — “if X happens, do Y” (see section 7).
- **Staff quick-card** and a short admin guide: menu, outlets, staff, refunds, reports.
- **This guide, the design & delivery plan, and the architecture notes.**
- **Repository README** — how to run, test, build and deploy.
- **Environment-settings sheet** — every setting name, what it does, where it lives (values stay in the vault).

Training sessions

AUDIENCE	COVERS	LENGTH
Owner / admins	Dashboard tour: menu, sold-out switches, outlets and hours, staff access, complaints and refunds, sales report; reading the Paystack dashboard; what to do in an incident.	~1.5 h
Counter staff	Live board, scanning, code fallback, customer problems.	~1 h

After handover: support and maintenance

Agree in writing: who answers when something breaks (hours and response times), how bugs and requests are raised, who applies updates, and how often. Apps need periodic upkeep — package security updates, annual Expo SDK upgrades, store policy changes, and new Android/iOS versions can each require a new build.

Done when: the client confirms every item above, signs the acceptance, and the hypercare period ends with no open must-fix issues.

7 · Running the app after launch

Every day (owner)

- Look at the staff board volume and any Ready orders waiting too long
- Open Complaints & refunds; resolve within the policy time
- Check yesterday’s sales on the Sales tab

Every week

- Match Paystack payouts with the Sales report
- Review slot fullness; adjust capacity
- Check SMS credit balance
- Skim error-monitoring for new issues

Every month / year

- Confirm backups; restore-test quarterly
- Apply package and security updates
- Renew Apple (yearly), domain, plans
- Review costs and the support arrangement

If something goes wrong

SYMPTOM	FIRST CHECKS
Customer paid but sees no order	1) Find the charge in the Paystack dashboard. 2) Check the webhook log there and resend it if it failed. 3) Ask the customer to open the order (the app re-checks with Paystack automatically). 4) If the pickup slot had expired, the system refunds automatically — confirm the refund in Paystack.

Login code doesn't arrive	SMS wallet balance and delivery report; try another network; confirm the sender ID is still approved; wait a minute before requesting again (the system limits repeat requests).
Staff can't scan	Dashboard must be on HTTPS; allow camera permission in the browser; use the order-code fallback so the customer isn't held up.
Refund stuck "pending"	Check the refund in Paystack; they confirm asynchronously. If it shows processed there but not in the dashboard, check the webhook log.
App or API down	The uptime monitor should alert first. Check the host's status page and logs; the database host's status; recent deployments (roll back the last one if it started right after a release).
Wrong price or sold-out item	Menu tab: edit the price, or toggle that outlet's availability. Existing orders keep the price they were placed at.
A staff member leaves	Staff tab → remove access; takes effect immediately.

8 · Risks and how to handle them

RISK	LIKELIHOOD	WHAT TO DO
Paystack approval delayed	Medium	Apply on day 1; submit complete documents; keep working on test keys; set the launch date <i>after</i> approval, not before.
OTP SMS not reaching all networks / DND numbers	Medium	Test on all four networks early (Phase 4); confirm the OTP route in writing; keep a second provider option; consider voice-call OTP later.
Store rejection or slow review	Medium	Account deletion, privacy policy and reviewer login done beforehand; one week of buffer; reply to reviewers same day.
New Google Play account testing requirement	Medium	Check current rules on day 1; recruit testers early; may add ~2 weeks.
Problems that only appear on real phones	High	This is why Phase 5 exists. Budget fix time; test an older Android phone.
Staff adoption at the counter	Medium	Training, quick-card, order-code fallback, owner present on the first days.
Wrong kitchen capacity settings	Medium	Start conservative; adjust daily during the soft launch.
Data-protection compliance	Medium	Lawyer reviews policy and registration duties before launch; account deletion built; photos and phone numbers kept only as long as needed.
Photos lost on redeploy	Certain until fixed	Phase 2 item 7 (cloud storage) — do not launch without it.
Single point of knowledge	Medium	Runbook, documented settings sheet, a second admin on every account, maintenance agreement.

Cost categories (get current quotes — none are fixed here)

TYPE	ITEMS
One-time	Google Play registration; legal review; counter devices; test phones; poster printing; development/launch effort.
Yearly	Apple Developer Program; domain name.
Monthly	Hosting and managed database; image storage; error-monitoring plan (if beyond free); Expo plan (if beyond free); support/maintenance.
Per use	Paystack fee on each payment (and on refunds, check their schedule); SMS per login code.

9 · Master checklist

Tick as you go. Owner letters: **C** client · **Y** you · **AI** Claude · **T** together.

Phase 0 — Decisions & content

- ☐ C · Logo files & brand confirmed
- ☐ C · Real Abuja outlets: addresses, hours, capacity
- ☐ C · Menu, prices, photos
- ☐ C · Refund & complaint policy
- ☐ C · Ordering rules
- ☐ C · Staff & admin list
- ☐ C · Support contact & launch date
- ☐ T · Hosting option chosen

Phase 1 — Accounts

- ☐ C · Paystack verification submitted
- ☐ C · SMS account + sender ID submitted
- ☐ C · Apple & Google accounts opened
- ☐ T · Repo under client GitHub org; PR merged
- ☐ C · Expo, hosting, DB, domain, storage
- ☐ C · Password-manager vault
- ☐ Y · Monitoring set up

Phase 2 — Hardening

- ☐ AI · Deploy configuration
- ☐ AI · CORS restricted
- ☐ AI · Cloud image storage
- ☐ AI · Account deletion
- ☐ AI · Privacy & terms pages (+ legal review)
- ☐ AI · Reviewer login
- ☐ AI · EAS build config
- ☐ AI · Monitoring & logging
- ☐ AI · OTP route for DND numbers
- ☐ AI · Runbook & staff card
- ☐ AI · Security/dependency pass

Phase 3 — Deploy

- ☐ Y · DB + backups
- ☐ Y · Secrets generated
- ☐ Y · API deployed (staging, prod)
- ☐ T · Domain + HTTPS
- ☐ Y · First admin created
- ☐ Y · Dashboard published (HTTPS)
- ☐ T · Real outlets & menu entered
- ☐ Y · Smoke test + backup restore

Phase 4 — Payments / SMS / push

- ☐ Y · Paystack test keys + webhook
- ☐ Y · All payment drills pass
- ☐ Y · OTP on 4 networks + DND
- ☐ Y · Push credentials; push received
- ☐ T · Live keys + live webhook
- ☐ T · One real payment + one real refund

Phase 5 — Builds & devices

- ☐ Y · Preview builds (Android & iOS)
- ☐ T · Device checklist passes on all phones
- ☐ C · Counter device + scanner test
- ☐ AI/Y · All must-fix bugs closed

Phase 6 — Soft launch

- ☐ T · Sample data removed; real data verified
- ☐ T · Staff trained
- ☐ Y · Invitees have the build
- ☐ T · Daily reviews for 2 weeks
- ☐ C · Capacity tuned
- ☐ C · Written “go”

Phase 7 — Store & public launch

- ☐ AI/C · Listing text & screenshots
- ☐ Y · Reviewer login enabled (then disabled)
- ☐ Y · Apple submitted & approved
- ☐ Y · Google submitted & approved
- ☐ T · Launch day; poster; announcement
- ☐ T · First 100 orders reconciled

Phase 8 — Handover

- ☐ T · Ownership verified on every account
- ☐ Y · Developer access reduced; secrets rotated
- ☐ T · Restore test witnessed
- ☐ T · Training delivered
- ☐ T · Runbook & docs handed over
- ☐ T · Maintenance agreement; acceptance signed

Appendix A · Settings reference

Set these in the host's settings page for the **API**. Values go in the password manager — never in Git. The server checks them at start-up.

SETTING	WHAT IT IS	PRODUCTION NOTES
<code>DATABASE_URL</code>	PostgreSQL connection string	Required
<code>JWT_ACCESS_SECRET</code>	Signs login sessions	32+ random characters, unique; production start-up refuses short, default or <code>change-me...</code> values
<code>OTP_SECRET</code>	Protects stored login codes	Same rules as above
<code>QR_SECRET</code>	Makes pickup QR codes unforgeable	Same rules as above. Changing it later invalidates every existing QR code.
<code>PAYSTACK_SECRET_KEY</code>	Paystack secret key (also verifies webhooks)	Test key on staging, live key on production; server rejects the placeholder
<code>PAYSTACK_EMAIL_DOMAIN</code>	Email domain used for the payer address Paystack requires	A domain the client owns; server rejects the <code>.invalid</code> default in production
<code>PUBLIC_API_URL</code>	Public https address of the API	Used for the post-payment return page
<code>APP_CALLBACK_URL</code>	App deep link after payment	Leave as <code>veejay://payment-complete</code> unless the app scheme changes
<code>SMS_PROVIDER</code>	<code>console</code> (testing) or <code>termii</code>	Must be <code>termii</code> in production — the server refuses to start with <code>console</code> because it would write login codes to the log
<code>TERMII_API_KEY</code> , <code>TERMII_SENDER_ID</code>	SMS credentials and sender name	Only when SMS is <code>termii</code>
<code>NODE_ENV</code>	Environment name	<code>production</code> turns on the safety checks. Use it only on the live system ; name the rehearsal copy <code>staging</code> — but still give staging its own random secrets
<code>PORT</code>	Port the API listens on	Most hosts set it automatically

Mobile app: `EXPO_PUBLIC_API_URL` (set in the EAS build profile). **Dashboard:** `VITE_API_URL` (set at build time).

Appendix B · Commands cheat-sheet

COMMAND	WHEN
<code>npm ci && npm run build</code> (in <code>backend</code>)	Host build step
<code>npm run migrate</code>	Apply database migrations (every deploy; safe to repeat)

<code>npm start</code>	Run the API in production
<code>node dist/create-admin.js 0803... "Name"</code>	Create/promote an admin in production
<code>npm test</code>	67 backend tests (needs a local PostgreSQL)
<code>VITE_API_URL=... npm run build</code> (in <code>admin</code>)	Build the dashboard for upload
<code>eas build --platform android ios --profile preview production</code>	Cloud-build the mobile app
<code>eas submit --platform ios android</code>	Send a production build to the store

Appendix C · What I need from you to start Phase 2 now

- ☐ Which hosting option (A, B or C) — so I write the right deploy files
- ☐ The dashboard address you'll use (for the CORS setting), even if provisional
- ☐ Which image-storage provider (any S3-compatible one is fine)
- ☐ Whether to build the menu import now (send the client's spreadsheet when it exists)
- ☐ A go-ahead to open a pull request for the Phase 2 work

Appendix D · Plain-English glossary

API	The server the app and dashboard talk to; it holds the rules and the database.
Webhook	Paystack calling our server to say "this payment succeeded". Verified with a secret signature.
OTP	One-time password — the 6-digit login code sent by SMS.
Migration	A versioned change to the database structure, applied the same way in every environment.
Staging	A private rehearsal copy of the whole system, using test money.
EAS / Expo	The cloud service that builds the iPhone and Android apps and sends them to the stores.
TestFlight / internal testing	Apple's and Google's official ways to give a test build to a few people before the public release.
Staged rollout	Releasing to a small percentage of users first, then widening if all is well.
DND	Do-Not-Disturb registry; some SMS routes don't deliver to numbers on it.
NDPA	Nigeria Data Protection Act — rules for handling personal data such as phone numbers and photos.

Durations, effort figures and approval times in this guide are planning estimates, not guarantees. Third-party requirements and fees (Paystack, SMS providers, Apple, Google, hosts) change — confirm each against the provider's current documentation before relying on it.